

MP1 Experiment Report

DASE7506 · 3036746155 Meng Weitao

Abstract

Using the course-provided WikiText-2 text and fixed BPE-2048 tokenizer, this experiment starts with a randomly initialized GPT and investigates the training recipe, model scale, regularization, probability ensembling, window-local copying, and n-gram statistics from the training corpus. The final model combines the calibrated probabilities of two GPTs, a causal local-copy distribution, and a 4-gram distribution with continuation-count smoothing. Full-test CPU FP32 **BPB is 1.458312825**, with validation BPB of **1.443548411**. Relative to the original baseline test BPB of 2.101256277, this is an absolute reduction of 0.642943, or approximately 30.60%.

Full-test scoring takes 51.424 seconds, or 4.756 times the baseline measured on the same machine during the same measurement period. The conservative upper bound on peak process-tree RAM is 2.471 GiB, and all uncompressed inference assets total 55.775 MiB. Local checks pass the limits of 5 times baseline CPU scoring time, 4 GiB RAM, and 64 MiB of inference assets. This report covers the progression from the initial baseline to the final frozen model, distinguishes controlled paired experiments from exploratory comparisons, and discloses search costs, historical test-set access, and AI assistance.

1. Task, Metrics, and Training and Evaluation Pipeline

The assignment seeks the lowest full-test bits per byte (BPB) reproducible with CPU FP32 evaluation within the resource limits. Training duration and model architecture may be changed, but learning must use only the supplied training text. Development and selection use validation data, and testing follows method freezing. The data, tokenizer, and evaluator must remain unchanged. External training text, pretrained weights, future tokens, cross-window state, cached validation/test answers, and network access during evaluation are prohibited. This experiment preserves the file hashes of the original `common.py`, `evaluate.py`, `model.py`, baseline configuration, all three text splits, tokenizer, and data manifest.

Training pipeline. The training text is encoded into a token sequence. Segments of length 257 are randomly sampled; the first 256 tokens form the input and the last 256 form the targets. Training uses causal attention and cross-entropy loss. With a batch size of 32, each optimizer update processes $32 \times 256 = 8,192$ prediction targets. AdamW is used with gradient-norm clipping at 1. The learning rate warms up for 100 steps and then follows a cosine schedule down to 10% of its peak. The number of processed targets, which includes repeated sampling, measures computational work rather than the amount of unique text.

Validation and testing pipeline. The scorer divides the data into independent windows. Each full window contains 256 prediction targets, and valid targets in the final partial window also contribute to the score. The predictor receives only input tokens and returns normalized natural-log probabilities over the vocabulary at each position; the scorer retains the reference targets. Checkpoints and probability parameters are selected using FP32 BPB on the full validation set. Final testing uses the unchanged scorer on CPU in FP32 with four threads.

Metric. $\text{BPB} = \text{total NLL} / \ln(2) / \text{raw UTF-8 byte count}$. The denominator is the byte count of the corresponding original text, not its token count. Final validation scores 376,599 targets over 1,148,007 bytes; testing scores 428,405 targets over 1,292,013 bytes. Unless explicitly labeled as test results, BPB values in the experimental tables below refer to validation.

2. Stage One: Baseline Reproduction and Short-Budget Optimization

The original GPT has a hidden width of 128, 4 layers, 4 attention heads, and 1,088,256 parameters. Training the original recipe for 1,200 steps with seed 17 in FP32 processes 9,830,400 targets and yields validation BPB of 2.071080 and test BPB of 2.101256. Initial short-budget experiments hold model size, random seed, and processed target count fixed while examining dropout, exponential moving average (EMA), weight decay, and learning rate.

Short-budget experiment (1,200 steps each)	Validation BPB	Observation relative to the original baseline
Original recipe: lr 0.001, wd 0.1	2.071080	Reference
Residual dropout 0.05	2.087937	No improvement
Residual dropout 0.1	2.102601	No improvement
EMA 0.99, enabled from step 100	2.075090	No improvement
Change only wd to 0.01	2.064279	Small improvement
Change only lr to 0.005	1.915215	Substantial improvement
lr 0.005, wd 0.01	1.907978	Selected for this stage

The learning-rate search covers 0.0015 to 0.006. The results indicate that optimization speed is an important bottleneck under the short budget. For the selected lr 0.005, wd 0.01 run, validation BPB at steps 300, 600, 900, and 1,200 is approximately 2.244, 2.090, 1.969, and 1.908, respectively, and is still decreasing at the end. This curve belongs to the tuned run and must not be attributed to the original training recipe.

A further short-budget check with seed 23 yields validation BPB of 2.076011 for the original recipe and 1.919401 for lr 0.005, wd 0.01, showing the same direction of improvement as seed 17. This check supports only the observation about the short-budget training recipe; it does not imply that the larger models or all probability mechanisms were evaluated across multiple seeds.

This stage shows that restricting the search to a small model trained for 1,200 steps limits attainable performance. This budget is the baseline reproduction setting, not an assignment-imposed training limit. The lack of benefit from dropout and EMA under this short budget does not imply that they are ineffective for larger models or longer training.

3. Stage Two: Model Scaling, Longer Training, and Probability Ensembling

The wide GPT increases the hidden width to 256 and uses 6 layers and 8 attention heads, for a total of 5,328,896 parameters. The table reports both the actual number of completed steps and the step of the selected checkpoint, so that selecting earlier weights is not confused with the full training cost.

Model and recipe	Completed steps / selected step	Validation BPB
Original-size GPT, lr 0.001, wd 0.1	8,000 / 8,000	1.703894
Wide GPT, no dropout, lr 0.0015, wd 0.01	8,000 / final step reported here	1.749122
Wide GPT, dropout 0.1, lr 0.0015, wd 0.1	8,000 / 8,000	1.565905
Wide GPT, dropout 0.1, lr 0.0015, wd 0.1	16,000 / 14,000	1.565567
Original-size GPT, lr 0.002, wd 0.01	12,000 / 11,000	1.678491
Probability ensemble of the wide and small models above, weights 0.69 / 0.31	Member weights from steps 14,000 / 11,000	1.518175

Each 8,000-step run processes 65,536,000 training targets. The regularized wide-model recipe improves BPB by 0.137988 over the original-size model, but model size, learning rate, and dropout change together. This is therefore a comparison of complete recipes at an equal target count, rather than evidence that model size alone accounts for the full improvement.

The wide model without dropout reaches approximately 1.651423 at step 4,000 before worsening to 1.749122 at the final step, consistent with overfitting. Adding dropout 0.1 and increasing weight decay improves the result. Since both settings change, this comparison does not isolate the contribution of dropout. The trainer subsequently saves the weights with the best validation score instead of saving the final-step weights by default.

Probability ensembling of the wide and small models further reduces validation BPB from 1.565567 to 1.518175. Results are similar when the small-model weight is approximately 0.29 to 0.325; the validation grid selects 0.31, indicating complementary predictions from the two members. The frozen model at this stage achieves test BPB of **1.539524**. This is an intermediate result; the final model submitted in this report achieves test BPB of **1.458313**.

The two complete member-training runs in Stage Two process 131,072,000 and 98,304,000 targets, totaling 229,376,000. The training ancestry of the selected weights at steps 14,000 and 11,000 accounts for 204,800,000 targets. The former measures search cost, while the latter describes the provenance of the weights selected at that stage.

4. Stage Three: Final Neural Members and Controlled Comparisons

The final selection is the continued-training wide model combined with a medium model. Both are causal GPTs with learned positional embeddings and dropout on residual branches. The medium model has width 160, 4 layers, 4 attention heads, and 1,606,080 parameters. Together, the two neural members contain 6,934,976 parameters.

Training run	Seed / peak lr / wd	Targets in this run	Validation BPB
Wide GPT, dropout 0.1, validation record at step 16,000	17 / 0.0015 / 0.1	131,072,000	1.567040
Wide GPT, dropout 0.2, trained for 16,000 steps	17 / 0.0015 / 0.1	131,072,000	1.529609
Continue the above dropout 0.2 wide model for 4,000 additional steps	23 / 0.0002 / 0.1	32,768,000	1.523762
Medium GPT, dropout 0.1, trained for 16,000 steps	23 / 0.002 / 0.1	131,072,000	1.600248

The first two rows both use FP32 and hold architecture, random seed, training target count, learning rate, and weight decay fixed, changing only residual dropout. Increasing dropout from 0.1 to 0.2 reduces validation BPB by **0.037431**, satisfying the assignment's requirements for an equal-target comparison and a paired control of a key mechanism. The dropout 0.1 result used here is the step-16,000 validation record, 1.567040. That run actually saved its best checkpoint at step 14,000, with validation BPB of 1.565567.

Subsequent low-learning-rate training of the wide model reduces BPB by approximately another 0.005846. However, it adds training and restarts the learning-rate schedule, so it is a comparison of continued-training recipes. The medium model is weaker on its own than the wide model, but still helps in the probability mixture. All three final training runs use CUDA FP32. The two wide-model runs validate every 1,000 steps, and the medium-model run validates every 2,000 steps. The selected checkpoint for each of these three runs is at that run's final step.

5. Final Probability Model and Mechanism Ablations

5.1 Neural probability calibration. Each member's logits are first divided by a temperature and then adjusted by a training-frequency bias. Let add-one smoothing of training token frequencies give $f(w) = (\text{count}(w) + 1) / (N_{\text{train}} + 2048)$, with centered bias $b(w) = \log f(w) - \text{mean}(\log f)$. Both members use temperature 1.1 and bias coefficient 0.05. The wide and medium models receive probability weights of 0.75 and 0.25, respectively:

```
p = 0.75 * softmax(z_wide / 1.1 + 0.05*b)
  + 0.25 * softmax(z_medium / 1.1 + 0.05*b)
```

This is a convex combination of probability distributions, not a direct average of logits. All these parameters are selected on validation data. Nearby temperature settings were explored, but the final frozen values are explicitly 1.1 / 1.1.

5.2 Window-local copying. For suffixes of lengths 1, 2, and 3 in the current input prefix, the method counts successor tokens already observed after matching contexts within that prefix. This yields empirical distributions q_1 , q_2 , q_3 and corresponding total observed successor counts n_1 , n_2 , n_3 . Statistics are updated only when the successor token is already part of the currently visible prefix. Neither the reference target at the current position nor future input is accessed. When there are no observations, the corresponding mixture weight is zero.

```
g1 = 0.05*n1/(n1+1) * (n2==0)
g2 = 0.4*n2/(n2+1)
g3 = 0.2*n3/(n3+1)
p_local = (1-g3)*((1-g1)*((1-g2)*p + g2*q2) + g1*q1) + g3*q3
```

This mechanism captures repeated names and phrases within the current window. Statistics are independent for each example and discarded after each call. No evaluation state is shared between examples in a batch or between windows.

5.3 Training-corpus 4-grams. Compact statistics are built from 3,613,343 tokens in the fixed training text, containing 298,024 / 1,391,944 / 2,410,077 distinct 2-grams / 3-grams / 4-grams. The raw uncompressed count asset occupies 30,522,560 bytes and is embedded in the checkpoint. Inference requires no external count file; the separate file used during construction is not counted again as a second inference copy.

The model uses interpolated continuation-count smoothing. The 4-gram model uses raw occurrence counts, while lower orders count distinct left contexts. The unigram distribution is based on the number of distinct predecessors, with a uniform pseudocount of total mass 1. Let $c(h, w)$ denote the count used at the current order, $N(h)$ the total successor count, and $T(h)$ the number of distinct successors. With discount $D=0.5$:

```
q(w|h) = max(c(h,w)-D, 0)/N(h)
         + D*T(h)/N(h) * q(w|suffix(h))
p_final = 0.875*p_local + 0.125*q4
```

Contexts absent from the training statistics back off completely to the lower-order distribution. All persistent count tables depend only on training assets, not on evaluation text. The final training-count weight is 0.125. The raw-count alpha field retained in the configuration is not used by continuation-count smoothing.

5.4 Incremental ablations. The following comparisons fix the same continued-training wide model and medium model, and use the full validation set to examine the incremental effects of the probability mechanisms. The final two rows jointly adjust the smoothing parameters and mixture weights.

Probability mechanism	Validation BPB
Calibrated probability mixture of the two neural models	1.482686
Add window-local copying	1.460861

Probability mechanism	Validation BPB
Add raw-count 4-grams, alpha 4, weight 0.1	1.447574
Adjust raw-count smoothing, alpha 0.5, weight 0.0875	1.445301
Use continuation-count discount D 0.5, weight 0.125	1.443548

Local copying reduces BPB by approximately 0.021825, indicating that repetition within the current window provides additional information. Training n-grams further contribute local statistics that the neural members have not fully retained. The gains from the final two steps are small, so statistical significance cannot be claimed from this validation result alone.

6. Rejected Methods, Implementation Corrections, and Resource Trade-offs

Architecture replacement. A RoPE/RMSNorm/SwiGLU variant with 5,344,512 parameters is trained for 8,000 steps with seed 17 in FP32, processing 65,536,000 targets. Its validation BPB is 1.581494, which does not beat the wide GPT recipe's 1.565905 at the same target count, so it is not adopted. This result applies to the tested recipe and does not establish that these architectural mechanisms are generally ineffective.

Stronger dropout and longer training. A run with dropout 0.3, BF16, and 24,000 steps processes 196,608,000 targets and achieves single-model validation BPB of 1.520760. Under fixed combination settings, it obtains 1.445744, worse than the final 1.443548, so it does not replace a final member. Since dropout, training precision, and duration all change, this is not a controlled ablation of dropout alone.

Other probability mechanisms. A continuous-feature cache helps when used alone with the earlier ensemble, but when combined with the stronger discrete cache it changes validation BPB from approximately 1.468334 to 1.469398, so it is rejected. Entropy-based member weighting improves BPB by only approximately 0.00043, while feature computation in the probe takes an additional 1.82 seconds. This is not a difference between independently measured total official-scorer times. The method is rejected after considering the extra computational cost. More complex count-dependent gating does not outperform a constant training 4-gram weight.

Correction of the early n-gram conclusion. An early raw-count probe inconsistently used integer and tuple keys in unigram lookups, producing an invalid poor score. That score and the resulting conclusion that n-grams were unhelpful are discarded. After the correction, the asset is rebuilt from the fixed training text. Target-probability probes are checked for consistency with the full normalized distributions, and selected candidates are verified using the unchanged full scorer. The old probe result therefore cannot be used to explain the final count model's effect.

CPU implementation optimization. Linear-layer weights are prepacked, and count updates and probability scaling are vectorized to avoid expensive repeated copies. These changes preserve the probability formulas, allowing small FP32 numerical differences. Prepacked weights, precomputed probabilities, and derived tables depend only on frozen training assets. Final prediction follows the CPU FP32 path. Neural modules fall back to standard PyTorch operations when gradients are required or when called on a non-CPU device. These performance optimizations allow the final combination to complete scoring within the local 5-times-baseline time limit.

7. Final Testing, Resource Measurements, and Compliance Checks

Frozen model at each stage	Validation BPB	Test BPB
Original GPT baseline	2.071080	2.101256
1,200-step tuning: lr 0.005, wd 0.01	1.907978	1.938087
Stage Two wide/small GPT probability ensemble	1.518175	1.539524
Final neural models + local copying + continuation-count 4-grams	1.443548411	1.458312825

Disclosure of test-set use. The test set was accessed more than once during the overall development process. In addition to the stage results above and baseline reruns, historical written records report test BPB of 1.540405 for a provisionally frozen ensemble; its standalone scoring JSON has not been found in the current archive. Subsequently, the existing validation curves were used to replace the small model's final-step weights with its best weights at step 11,000. The ensemble weights were reselected on validation data, and the newly frozen model achieved the Stage Two result of 1.539524. Experiment records indicate that model and parameter selection used validation results and that historical test scores were not used as the selection objective. Nevertheless, those scores had been observed, so the final test should not be described as the project's first exposure to an independent held-out set. The final method and checkpoint were frozen before this round of full-test evaluation, and the predictor was not adjusted after obtaining 1.458313.

Final resource or protocol item	Measurement / check result	Assignment limit
Full-test CPU FP32 scoring time	51.423691 s; contemporaneous baseline 10.812416 s	Ratio 4.755985, below 5 times
Conservative upper bound on peak evaluation RAM	2.470680 GiB	At most 4 GiB
All uncompressed inference assets	58,484,077 bytes, or 55.774762 MiB	At most 64 MiB
Full-test scoring coverage	428,405 targets; 1,292,013 bytes	Original scoring protocol
Normalization, causality, independence, and reset	Seven provided tests and actual-checkpoint checks pass	No future information or cross-window state
Data, tokenizer, and original scorer	Hashes unchanged	Must remain unchanged

Timing measurements are performed sequentially on the same computer with four threads, after stopping training and other Python probes. Both models use the original scorer's timed region, excluding loading from CPU scoring time. The baseline timing in the earlier Stage Two report was measured previously and cannot replace the contemporaneous baseline measurement in the table above.

RAM measurement covers the Windows measurement runner, Python launcher, and scorer descendants, from imports and loading through scoring and result saving. The sum of each process's historical peak working set is a conservative upper bound, not necessarily the peak total RAM at a single instant. The report uses the larger bound from the full validation and test measurements. GPU memory is not substituted for evaluation RAM.

Inference-asset accounting includes the checkpoint, required Python inference files, fixed tokenizer, and manifest, using uncompressed sizes rather than ZIP sizes. The checkpoint itself contains 58,323,858 bytes, including both neural members and the raw count bytes. Course-provided benchmark input text and general runtime libraries are not learned inference assets.

The frozen checkpoint and all inference-dependency hashes are checked before and after scoring. The actual checkpoint passes checks for finite values, probability normalization, causality, independence within a

batch, reset across repeated calls, short prefixes, and gradients. It also loads and predicts in an isolated directory containing only the checkpoint and required inference code, without external n-gram files or analysis probability arrays. These findings are local measurements, not guarantees of identical timing ratios on other hardware. In particular, 4.756 times baseline is already close to the limit of 5 times.

8. Training Costs and Limitations

The training ancestry of the final neural members processes **294,912,000** targets: 131,072,000 for initial wide-model training, 32,768,000 for continued training, and 131,072,000 for the medium model. Shared ancestors are counted only once. Recorded training times for the three runs are approximately 1,677.075, 484.301, and 770.293 seconds, totaling approximately 2,931.669 seconds. Count construction separately reads 3,613,343 training tokens; it is not an optimizer update and must not be described as additional training text.

Cost-accounting scope	Processed training targets	Evidential basis
Training ancestry of the final selected neural models	294,912,000	Verifiable from three run metrics
Five completed Stage Three runs, including rejected candidates	557,056,000	Verifiable from completed-run metrics
All retained completed-run metrics, including repeated baselines and the environment smoke test	1,268,203,520	Verifiable from 31 run-metrics records
Also include at least 4,000 recorded steps of a historical interrupted continuation	At least 1,300,971,520	Lower bound dependent on the retained ledger
Estimate that interrupted continuation at approximately 4,500 steps	Approximately 1,305,067,520	Nominal total, not an exact measurement

The historical interrupted continuation lacks final metrics and a child checkpoint, so the estimate cannot be stated as an exact total. The cost of a complete training run cannot be calculated only from an earlier selected checkpoint. Assembling a probability ensemble requires no new optimizer updates, and member ancestry must not be counted twice. The table reports processed training targets, not the total time spent on all validation grids, CPU tests, and asset construction; it must not be interpreted as complete end-to-end computation time.

Repeated architecture and probability-parameter selection increases the risk of overfitting the validation set, and small validation improvements may not generalize to test data. Most comparisons use only one random seed and do not estimate variance across independent repetitions. Equal target counts control the number of processed training examples, not FLOPs or training time. Historical test results have already been observed, which is another limitation that must be retained when interpreting the final result.

Final test BPB of **1.458313** is close to the exploratory target of approximately 1.45, but is not at or below 1.450000. The lower validation BPB of 1.443548 must not be submitted as the test score, nor does it establish that 1.4 is achievable. The final approach provides a reproducible local trade-off between prediction quality and resource cost; optimization has ended.

9. Reproducibility and Evidence Locations

Environment. The local environment uses Windows, Python 3.12, PyTorch 2.7.1+cu126, NumPy 2.5.3, and tokenizers 0.21.4. Training uses an NVIDIA GeForce RTX 3050 Ti Laptop GPU, while final scoring uses CPU FP32 with four threads. The CPU build of PyTorch 2.7.1 can be used to reproduce scoring alone. Run the following commands from `MP1_student_starter/code/` in the code bundle; relative paths refer to the `experiments/` directory at the bundle root.

```
python -m pip install torch==2.7.1 --index-url https://download.pytorch.org/whl/cpu
python -m pip install numpy==2.5.3 tokenizers==0.21.4
python -m unittest discover -s tests -v
python verify_hybrid.py --checkpoint ../../experiments/final-hybrid-v3/checkpoint.pt
python evaluate.py --checkpoint ../../experiments/final-hybrid-v3/checkpoint.pt --device cpu --precision
fp32 --threads 4 --split test
```

If the checkpoint is extracted elsewhere, adjust only its path. Direct scoring requires no retraining, external count files, or network requests. The released checkpoint's SHA256 is:

```
5b99639aa30264a10ec0d9a7608317fec5e926b521ce205b64c2b9bfde1183e1
```

Reconstruction recipe. The three final training runs share the arguments `--implementation student --device cuda --precision fp32 --threads 4 --batch-size 32`; other settings are given in Section 4. The wide-model configuration is `configs/scaled_256x6_dropout02.json`, and the medium-model configuration is `configs/scaled_160x4_dropout01.json`. First train the wide model for 16,000 steps, then pass its best checkpoint as `--init-checkpoint` for another 4,000 steps. Independently train the medium model for 16,000 steps. Use separate fresh output directories, validate at the frequencies specified in Section 4, and save the best weights.

Next, run `build_ngram_asset.py` to reconstruct counts from the fixed training text. Use `select_hybrid.py` with the single fixed parameter setting in Section 5 to generate a selection record with matching paths and hashes, and then assemble the checkpoint using `build_hybrid.py`. Reconstructing a fixed configuration is not a new search. The raw count asset's SHA256 is:

```
1c318f7fb8669d9bacba4a73be691bb9deab1fad5735c7a90ee2a55e475a5d09
```

The code bundle's `FINAL_REPRODUCTION.md` provides complete, copyable commands for training, fixed model assembly, and RAM measurement. Retraining on another platform may produce different checkpoint bytes; changes to serialization metadata can also change the hash. Exact identity is defined by the frozen checkpoint listed in this report, with reasonable FP32 numerical differences allowed in direct scoring.

Evidence files. Final results and checks are stored under `experiments/final-hybrid-v3/` in `validation_cpu_fp32.json`, `test_cpu_fp32.json`, `resource_measurements.json`, `contract_checks.json`, `portability_checks.json`, `FREEZE.json`, and `COMPLIANCE.json`. Contemporaneous baseline records are in `experiments/final-compliance-baseline/`. Early search records are in `experiments/experiment_log.csv`, and cost accounting is in `experiments/phase3_training_ledger.json` and `experiments/additional_training_cost_audit.json`. The report itself contains the methods, key comparisons, and conclusions; these files support verification of individual runs.

10. AI Assistance and Reuse Disclosure

AI assistance. OpenAI Codex assisted with checking the Python environment and converting scripts, implementing model and evaluation support code, debugging, and designing and running experiments.

Reuse disclosure. The base GPT, training and evaluation framework, data processing, and fixed tokenizer come from the course-provided starter; the extensions build on that foundation. The original data and evaluation protocol remain unchanged. No external training text or pretrained weights are used. The course source and AI assistance are also disclosed in the repository README.